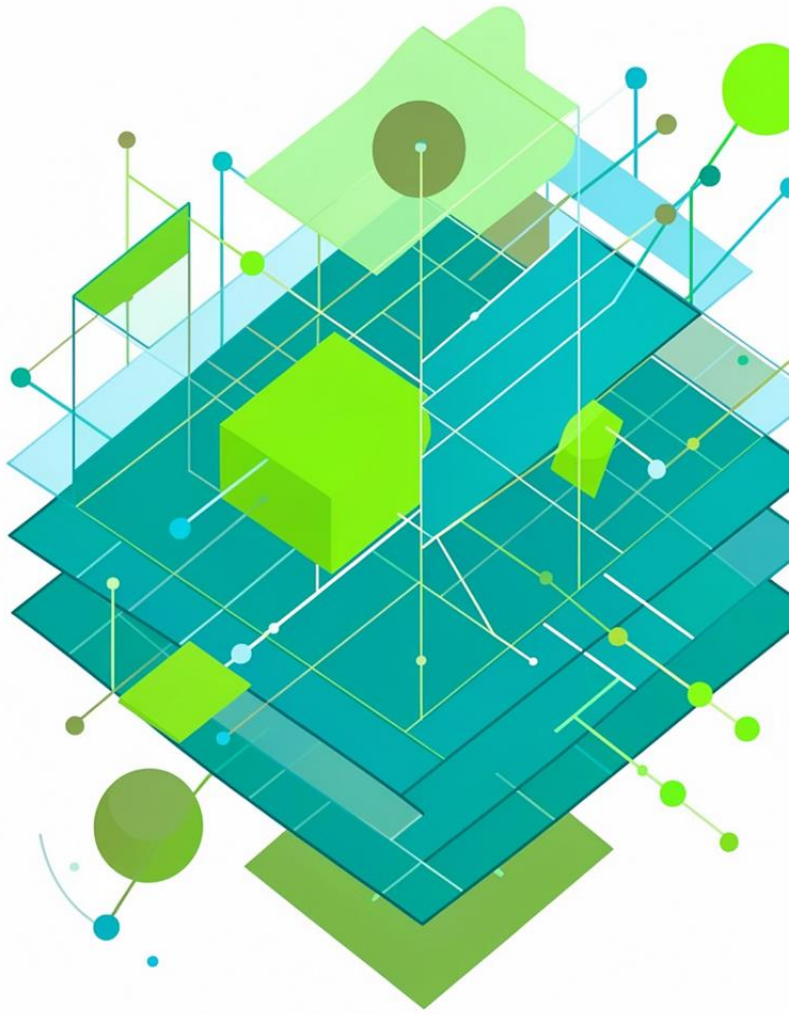
NumPy and Pandas

Presented by
Vasudeva Shenoy U
Assistant Professor,
Dept. Of Computer Science, YIASCM, Balmatta.



NumPy: Numerical Python

- NumPy stands for ‘Numerical Python’.
- It is a package for data analysis and scientific computing with Python.
- NumPy uses a multidimensional array object and has functions and tools for working with these arrays.
- The powerful n-dimensional array in NumPy speeds-up data processing.
- NumPy can be easily connected with other Python packages and provides tools for integrating with other programming languages like C, C++ etc.

Array

- An array is a data type used to store multiple values using a single identifier (variable name).
- An array has an ordered collection of data elements where each element is of the same type and can be referenced by its index (position).

Characteristics of an Array

- Each element of the array is of same datatype, though the values stored in them may be different.
- The entire array is stored contiguously in memory.
- Each element of the array is found or referred using the name of the Array along with the index of that element, which is unique for each element.

Characteristics of an Array

- The index of an element is an integral value associated with the element, based on the element's position in the array.
- For example, consider an array with 5 numbers:

[10, 9, 99, 71, 90]

Here, the 1st value in the array is 10 and has the index value [0] associated with it; the 2nd value in the array is 9 and has the index value [1] associated with it, and so on. The last value (in this case the 5th value) in this array has an index [4].

NumPy Array

- NumPy arrays are used to store lists of numerical data, vectors and matrices.
- The NumPy library has a large set of routines (built-in functions) for creating, manipulating, and transforming NumPy arrays.
- The NumPy array is officially called ndarray but commonly known as array.

How to create a Numpy array?

array() : To create a 1D/2D array, we can simply pass a list to the array method.

```
In [1]: # To create a 1-D array
        import numpy as np
        a1 = np.array([1,2,3,4,5])
        print(a1)
```

```
Out[1]: [1 2 3 4 5]
```

(1-Dimensional Array)

```
In [2]: # To create a 2-D array
        import numpy as np
        a1 = np.array([[1,2,3],[4,5,6]])
        print(a1)
```

```
Out[2]: [[1 2 3]
         [4 5 6]]
```

(2-Dimensional Array)

How to create a Numpy array?

empty() : It create a empty array or an uninitialized array of specified shape and dtype.

Syntax: <arrayname> = numpy.empty (Shape, [dtype = <data type>,] [order = 'C' or 'F'])
where,
Shape: is dimension. **dtype**: is data type (optional).
Order: 'C' means column-wise and 'F' means Fortran-like arrangement of data (optional).
Default data type and Order are float and 'C' respectively.

```
In [1]: # To create an empty array
        import numpy as np
        arr = np.empty([3,2], dtype = int)
        print(arr)
```

How to create a Numpy array?

zeros() : This function works in the same manner as empty(), but returns a new array with all zeroes.

Syntax: <arrayname> = numpy.zeros (shape, [dtype = float,] [order = 'C'/'F'])
where,
Shape: int or tuple of ints.
dtype: is data type (optional). Default is float64.
Order: 'C' (row-major) or 'F' (column-major), optional. Default is 'C'.

```
In [1]: # array of five zeros.
        # Default dtype is float
        import numpy as np
        arr = np.zeros([5], dtype = int)
        print(arr)
```

```
Out[1]: [0 0 0 0 0]
```

How to create a Numpy array?

ones() : This function works in the same manner as empty(), but returns a new array with all ones.

Syntax: <arrayname> = numpy.ones (shape, [dtype = float,] [order = 'C'/'F'])
where,
Shape: int or sequence of ints.
dtype: data type / optional. Default is float64.
Order: 'C' (row-major) or 'F' (column-major), optional. Default is 'C'.

```
In [1]:# array of five ones.
       # Default dtype is float
       import numpy as np
       arr = np.ones([5])
       print(arr)
```

```
Out[1]:[1. 1. 1. 1. 1.]
```

How to create a Numpy array?

arange() : It is used to create array from a range. This method takes the start index of the array, the end index and the step size (optional).

Syntax: <arrayname> = numpy.arange ([start], stop, [step], [dtype])

```
In [1]: # Creation of array using arange()
        import numpy as np
        arr = np.arange(1,10,2)
        print(arr)
```

```
Out[1]: [1 3 5 7 9]
```

How to create a Numpy array?

linspace() : This function can be used to prepare array of range. It returns evenly spaced numbers over a specified interval.

Syntax: <arrayname> = numpy.linspace ([start], stop, num, [dtype])

Parameters:
start: starting value of the sequence;
num: number of samples to generate;
stop: End value of the sequence
dtype: data type

```
In [1]: # Creation of array using linspace()
        import numpy as np
        arr = np.linspace(1,10,5)
        print(arr)
```

How to create a Numpy array?

copy() : This function is used to create a copy of an existing array.

Syntax: <arrayname> = numpy.copy (<array>, [order = 'C'/'F'])

Parameters:
<array>: an existing array; **order:** 'C' (row-major) or 'F' (column-major), optional.

In [1]:
```
# Create copy of existing array.
import numpy as np
arr = np.array([10,20,30])
print(arr)
arr1 = arr # assigning arr to arr1
arrc = np.copy(arr) # copying arr into arrc
arr[0] = 100 # replaced the first element by 100
print(arr)
print(arr1)
print(arrc)
```

Out[1]:
```
arr before change =  [10 20 30]
arr =  [100  20  30]
arr1 =  [100  20  30]
arrc =  [10 20 30]
```

How to create a Numpy array?

reshape() : Gives a new shape to an array without changing its data.

Syntax: <arrayname> = numpy/[<array>].reshape ([<array>], <new shape>)

Parameters:
<array>: an existing array; **newshape:** new shape in the form of tuple.

In [1]:
```
# Creating 2D array from 1D existing array using reshape().
import numpy as np
arr = np.array([10,20,30,40,50,60])
arr1 = np.reshape(arr, (2,3)) # reshape in 2,3 2-D array.
print(arr1)
```

Out[1]:
```
[[10 20 30]
 [40 50 60]]
```

How to create a Numpy array?

eye()/identity() : The eye method can be used to create an identity matrix, which can be very useful to perform a variety of operations using arrays. An identity matrix is a matrix with zeros across rows and columns except the diagonal. It contains all 1s as its diagonal elements.

Syntax: numpy.identity (n, dype = None) / numpy.eye (n, dtype = float)

In [1]:
```
# Create a 4x4 identity matrix using the eye method.
import numpy as np

# 4x4 matrix with 1's as all diagonal elements
mat1 = np.identity(4)
print("\nMatrix 1 : \n", mat1)

# Alternatively
mat2 = np.eye(4, dtype = float)
print("\nMatrix 2 : \n", mat2)
```

Out[1]:
```
Matrix 1 :
 [[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

Matrix 2 :
 [[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

Accessing elements using Numpy array Indexing

Every element of an array will have an index. The dimensions in index is equal to the dimension of the array. i.e. if an array is 1–D then index of every element will have one index and if an array is 2–D then index of every element will two dimensions in index and so on. An attempt to access an element which is not present in the index range will results in error.

```
In [1]: import numpy as np
        arr = np.array([1,2,3,4,5,6,7,8,9,10])
        arr[5]
```

Accessing element of 1-D array

```
Out[1]: 6
```

```
In [2]: import numpy as np
        arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
        arr[0,2]
```

Accessing element of 2-D array

```
Out[2]: 3
```

Sorting of an array

Sorting is to arrange the elements of an array in ascending or descending order. By default, numpy does sorting in ascending order. In 2–D array, sorting can be done along either of the axes i.e., row–wise or column–wise.

Syntax: numpy.sort(ary, axis = None)

where, ary - array and axis is optional and can have 0/1 as value.

Sorting of an array

In [1]:
```
# Sorting of array
import numpy as np
a = np.array([[9,5,8],[4,2,1]])

print("Original Array:")
print(a)

print("Sort along the last axis:")
print(np.sort(a))

print("Sort the flattened array:")
print(np.sort(a, axis=None))

print("Sort along the first axis:")
print(np.sort(a, axis = 0))

print("Sort along the second axis:")
print(np.sort(a, axis = 1))
```

Out[1]:
```
Original Array:
[[9 5 8]
 [4 2 1]]
Sort along the last axis:
[[5 8 9]
 [1 2 4]]
Sort the flattened array:
[1 2 4 5 8 9]
Sort along the first axis:
[[4 2 1]
 [9 5 8]]
Sort along the second axis:
[[5 8 9]
 [1 2 4]]
```

Transposing of an array

The transpose( ) method or T attribute transposes the matrix means, converts rows into columns

Syntax: numpy.transpose(ary, axis = None)
or
<ary>.T

In [1]:
```
import numpy as np
a = np.array([[1,2,3,4],[5,6,7,8]])
print("Array before transpose :")
print(a)
print("Array after Transpose: ")
print(np.transpose(a))
print("Transpose by T attribute: ")
print(a.T)
```

Out[1]:
```
Array before transpose :
[[1 2 3 4]
 [5 6 7 8]]
Array after Transpose:
[[1 5]
 [2 6]
 [3 7]
 [4 8]]
Transpose by T attribute:
[[1 5]
 [2 6]
 [3 7]
 [4 8]]
```

Numpy array Slicing

Slicing means that a subsequence of the structure can be indexed and retrieved. It is specified by using the colon operator ':' . The slice extends from the 'from' index and ends one item before the 'to' index.

Syntax : data[from : to]

Example 1: Slicing in 1 D array.

```
In [1]: import numpy as np
        arr = np.array([1,2,3,4,5,6,7,8,9,10])
        print(arr[:]) #extracts slice from start to end
        print(arr[1:4]) #extracts slice from 1st index to 3rd index.
        print(arr[:2]) #extracts slice from start to 1st index, excluding 2nd index
        print(arr[-5:-2]) #extracts slice from index -5 to index -3. -2 excludes.
```

```
Out[1]: [ 1  2  3  4  5  6  7  8  9 10]
        [2 3 4]
        [1 2]
        [6 7 8]
```

Numpy array Slicing

Example 2: Slicing in 2 D array.

In [1]:
```
import numpy as np
a = np.array([(1,2,3,4),(5,6,7,8),(9,10,11,12)])
print(a[0:,3]) #Include all the rows but display only the element at index 3 in each row.
print(a[0:2,3]) #Include row at index 0 and 1 but display only the element at index 3 in each row.
print(a[1:2,1:2]) #slice constists of 2nd row, 2nd column
print(a[:2,1:3]) #slice constists of 2 rows first[2,3] and second[6,7]
print(a[:2, :3]) #print elements of 0,1 rows and 0,1,2 columns
print(a[:3, ::2]) #print elements of 0,1,2 rows and alternate column position
print(a[::-1, ::-1]) #print elements in reverse order
print(a[:,0]) #print all elements of 0 column
print(a[0,:]) #print all elements of 0 row
```

Out[1]:
```
[ 4  8 12]
[4 8]
[[6]]
[[2 3]
 [6 7]]
[[1 2 3]
 [5 6 7]]
[[ 1  3]
 [ 5  7]
 [ 9 11]]
[[12 11 10  9]
 [ 8  7  6  5]
 [ 4  3  2  1]]
[1 5 9]
[1 2 3 4]
```

Arithmetic Operations on arrays

Python Numpy module provides various arithmetic functions such as add, subtract, multiply and divide, which performs Python numpy arithmetic operations on arrays. Apart from them, you can use the standard Python Arithmetic Operators also. Arrays should be of the same shape, or they have to bound to array rules to use Numpy arithmetic functions.

Using Scalars:

```
In [1]:# Scalar Operations
       import numpy as np
       x = np.arange(8)
       print(x) # prints original array.
       print(x + 2) # Addition
       print(x - 2) # Subtraction
       print(x * 2) # Multiplication
       print(x / 2) # Division
       print(x % 2) # Modulus
       print(x ** 2) # Exponentiation
       print(x // 2) # Floor division
```

```
Out[1]:[0 1 2 3 4 5 6 7]
       [2 3 4 5 6 7 8 9]
       [-2 -1  0  1  2  3  4  5]
       [ 0  2  4  6  8 10 12 14]
       [0.  0.5 1.  1.5 2.  2.5 3.  3.5]
       [0 1 0 1 0 1 0 1]
       [ 0  1  4  9 16 25 36 49]
       [0 0 1 1 2 2 3 3]
```

Arithmetic Operations on arrays

Arithmetic Operations on 2 arrays by using operators and equivalent functions:
(Similarly it can be done on 2-D arrays also.)

In [1]:
```
# Arithmetic Operations
import numpy as np
x = np.array([1,2,3,4])
y = np.array([5,6,7,8])

print(x + y) # Addition
print(y - x) # Subtraction
print(x * y) # Multiplication
print(y / x) # Division
print(y % x) # Modulus
print(x ** y) # Exponentiation
print(y // x) # Floor division
```

Out[1]:
```
[ 6  8 10 12]
[4 4 4 4]
[ 5 12 21 32]
[5.         3.         2.33333333 2.        ]
[0 0 1 0]
[    1    64  2187 65536]
[5 3 2 2]
```

In [2]:
```
# Arithmetic Operations
import numpy as np
x = np.array([1,2,3,4])
y = np.array([5,6,7,8])

print(np.add(x,y)) # Addition
print(np.subtract(y,x)) # Subtraction
print(np.multiply(x,y)) # Multiplication
print(np.divide(y,x)) # Division
print(np.mod(y,x)) # Modulus
print(np.power(x,y)) # Exponentiation
print(np.floor_divide(y,x)) # Floor division
```

Out[2]:
```
[ 6  8 10 12]
[4 4 4 4]
[ 5 12 21 32]
[5.         3.         2.33333333 2.        ]
[0 0 1 0]
[    1    64  2187 65536]
[5 3 2 2]
```

Aggregate Operations on arrays

In the Python NumPy module, we have many aggregate functions or statistical functions to work with a single-dimensional or multidimensional arrays.

In [1]:
```
# Aggregate Operations
import numpy as np
a = np.array([10,20,30,40,50])
b = np.array([[10,20,30],[40,50,60]])

print(a.sum()) # calculates the sum of values in an array.
print(b.sum(axis=0)) # calculate the sum of a given axis.
print(a.max()) # returns the maximum value in an array or a given axis.
print(b.max(axis=0))
print(a.min()) # returns the minimum value in an array or a given axis.
print(b.min(axis=0))
print(np.average(a)) # returns the average of a given array or given axis.
print(np.average(b,axis = 0)) # axis = 0 means column wise.
print(np.prod(a)) #finds the product of all the elements in a given array or axis.
print(np.prod(b, axis = 1)) # axis 1 means row wise.
```

Out[1]:
```
150
[50 70 90]
50
[40 50 60]
10
[10 20 30]
30.0
[25. 35. 45.]
12000000
[  6000 120000]
```

Measuring Central Tendencies in arrays

NumPy provides functions to find mean and median operations on arrays. To find mode, we have to use mode function available in scipy library.

In [1]:

```
import numpy as np
from scipy import stats

list1 = [1,2,3,4,5,7,8,9,10]

mean = np.mean(list1)
medium = np.median(list1)
mode = stats.mode(list1)

print('Mean is ', mean)
print('Median is ', medium)
print('Mode is ', mode[0])

list2 = [1,2,3,4,5,7,7,8,9,10]

mean = np.mean(list2)
medium = np.median(list2)
mode = stats.mode(list2)

print('\nMean is ', mean)
print('median is ', medium)
print('Mode is ', mode[0])
```

Note: The mode function is not available in numpy library. So, to find the mode, we have to import stats module of scipy library.
To use scipy library, we have to install it first by the command given below in the linux terminal or windows command prompt:

```
pip3 install scipy
```

Out[1]:

```
Mean is  5.444444444444445
Median is  5.0
Mode is  [1]

Mean is  5.6
median is  6.0
Mode is  [7]
```

Measuring Variability in arrays

The Python NumPy var() function returns the variance of a given array or in a given axis.
The Python NumPy std() function returns the standard deviation of a given array or in a given axis.

In [1]:

```
# Standard Deviation & Variance
# in One dimentional Array.
import numpy as np
a = np.array([5, 8, 9, 4, 7, 3])

vr = a.var() # Calculates Variance
sd = a.std() # Calculates Standard Deviation

print('Variance is ', vr)
print('Standard Deviation is ', sd)
```

Out[1]:

```
Variance is  4.666666666666667
Standard Deviation is  2.160246899469287
```

In [2]:

```
# Standard Deviation & Variance
# in Multidimentional Array.
import numpy as np
a = np.array([[1, 2, 3, 4],
              [5, 5, 6, 6],
              [7, 8, 9, 10],
              [5,5,6,6]])

vr = a.var() # Calculates Variance
sd = a.std() # Calculates Standard Deviation

print('Variance is ', vr)
print('Standard Deviation is ', sd)
```

Out[2]:

```
Variance is  5.25
Standard Deviation is  2.29128784747792
```

Pandas:

- One of the most widely used Python libraries in Data Science
- The Pandas library Provides
  - High-performance
  - Easy-to-use data structures and
  - Data analysis tools

Pandas - Data Structures

- **Series objects**
  1D array, similar to a column in a spreadsheet

- **DataFrame objects**
  2D table, similar to a spreadsheet

- **Panel objects**
  Dictionary of DataFrames

Pandas - Series Objects

Creating a Series

```
import pandas as pd
s = pd.Series([2,-1,3,5])
```

Output

```
2
-1
3
5
```

Pandas - Series Objects

Pass as parameters to NumPy functions

```
import numpy as np
np.square(s)
```

Output

```
4
1
9
25
```

Pandas - Series Objects

Arithmetic operation on the series

s + [1000,2000,3000,4000]

Output

1002
1999
3003
4005

Pandas - Series Objects

Broadcasting

```
s + 1000
```

Output

```
1002
 999
1003
1005
```

Pandas - Series Objects

Binary and conditional operations

s < 0

Output

False
True
False
False

Pandas - Series Objects

Index labels - Integer location

```
s2 = pd.Series([68, 83, 112, 68])
print(s2)
```

Output

```
68
83
112
68
```

Pandas - Series Objects

Index labels - Set Manually

```
s2 = pd.Series([68, 83, 112, 68],
index=["alice", "bob", "charles", "darwin"])
print(s2)
```

Output

```
alice 68
bob 83
charles 112
darwin 68
```

Pandas - Series Objects

Access the items in series

- By specifying integer location

s2[1]

- By specifying label

s2["bob"]

Pandas - Series Objects

Access the items in series - Recommendations

- Use the loc attribute when accessing by label

s2.loc["bob"]

- Use iloc attribute when accessing by integer location

s2.iloc[1]

Pandas - Series Objects

Init from Python dict

```
weights = {"alice": 68, "bob": 83, "colin": 86,"darwin": 68}
s3 = pd.Series(weights)
print(s3)
```

Output

```
alice 68
bob 83
colin 86
darwin 68
```

Pandas - Series Objects

Control the elements to include and specify their order

```
s4 = pd.Series(weights, index = ["colin", "alice"])
print(s4)
```

Output

```
colin 86
alice 68
```

Pandas - Series Objects

Automatic alignment

- When an operation involves multiple Series objects
- Pandas automatically aligns items by matching index labels

Pandas - Series Objects

Automatic alignment - example

```
print(s2+s3)
```

Output

```
alice 136.0
bob 166.0
charles NaN
colin NaN
darwin 136.0
```

* Note NaN

Pandas - Series Objects

Automatic alignment

Do not forget to set the right index labels, else you may get surprising
results

```
s5 = pd.Series([1000,1000,1000,1000])
print(s2 + s5)
```

Output

```
alice NaN
bob NaN
charles NaN
darwin NaN
0 NaN
```

Pandas - Series Objects

Init with a scalar

```
meaning = pd.Series(42, ["life","universe","everything"])
print(meaning)
```

Output

```
life 42
universe 42
everything 42
```

Pandas - Series Objects

Series name - A Series can have a name

```
s6 = pd.Series([83, 68], index=["bob",
"alice"],name="weights")
print(s6)
```

- Here series name is weights

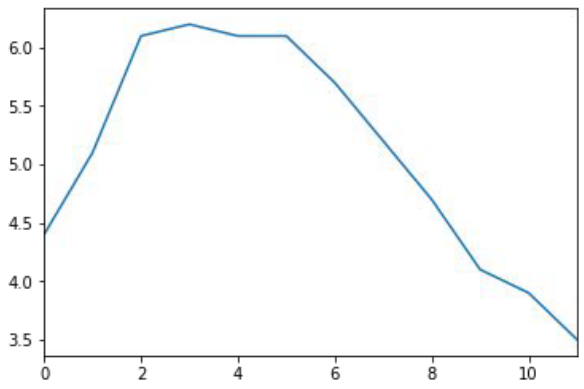
Output

```
bob 83
alice 68
Name: weights
```

Pandas - Series Objects

Plotting a series

```
%matplotlib inline
import matplotlib.pyplot as plt
temperatures
=[4.4,5.1,6.1,6.2,6.1,6.1,5.7,5.2,4.7,4.1,3.9,3.5]
s7 = pd.Series(temperatures, name="Temperature")
s7.plot()
plt.show()
```



Pandas - DataFrame Objects

- **A DataFrame object represents**
  - A spreadsheet,
  - With cell values,
  - Column names
  - And row index labels

Pandas - DataFrame Objects

Creating a DataFrame - Pass a dictionary of Series objects

```
people_dict = {
"weight": pd.Series([68, 83, 112],index=["alice","bob", "charles"]),
"birthyear": pd.Series([1984, 1985, 1992],
index=["bob", "alice", "charles"], name="year"),
"children": pd.Series([0, 3], index=["charles","bob"]),
"hobby": pd.Series(["Biking", "Dancing"],
index=["alice", "bob"]),
}
```

Pandas - DataFrame Objects

Creating a DataFrame

```
people = pd.DataFrame(people_dict)
people
```

	birthyear	children	hobby	weight
alice	1985	NaN	Biking	68
bob	1984	3	Dancing	83
charles	1992	0	NaN	112

Important Notes

- The Series were automatically aligned based on their index
- Missing values are represented as NaN
- Series names are ignored (the name "year" was dropped)

Pandas - DataFrame Objects

DataFrame - Access a column

```
people["birthyear"]
```

Output

```
alice 1985
bob 1984
charles 1992
Name: birthyear
```

Pandas - DataFrame Objects

DataFrame - Access the multiple columns

people[["birthyear", "hobby"]]

	birthyear	hobby
alice	1985	Biking
bob	1984	Dancing
charles	1992	NaN

Pandas – CSV File

A CSV is a comma separated values file, which allows data to be saved in a tabular format.

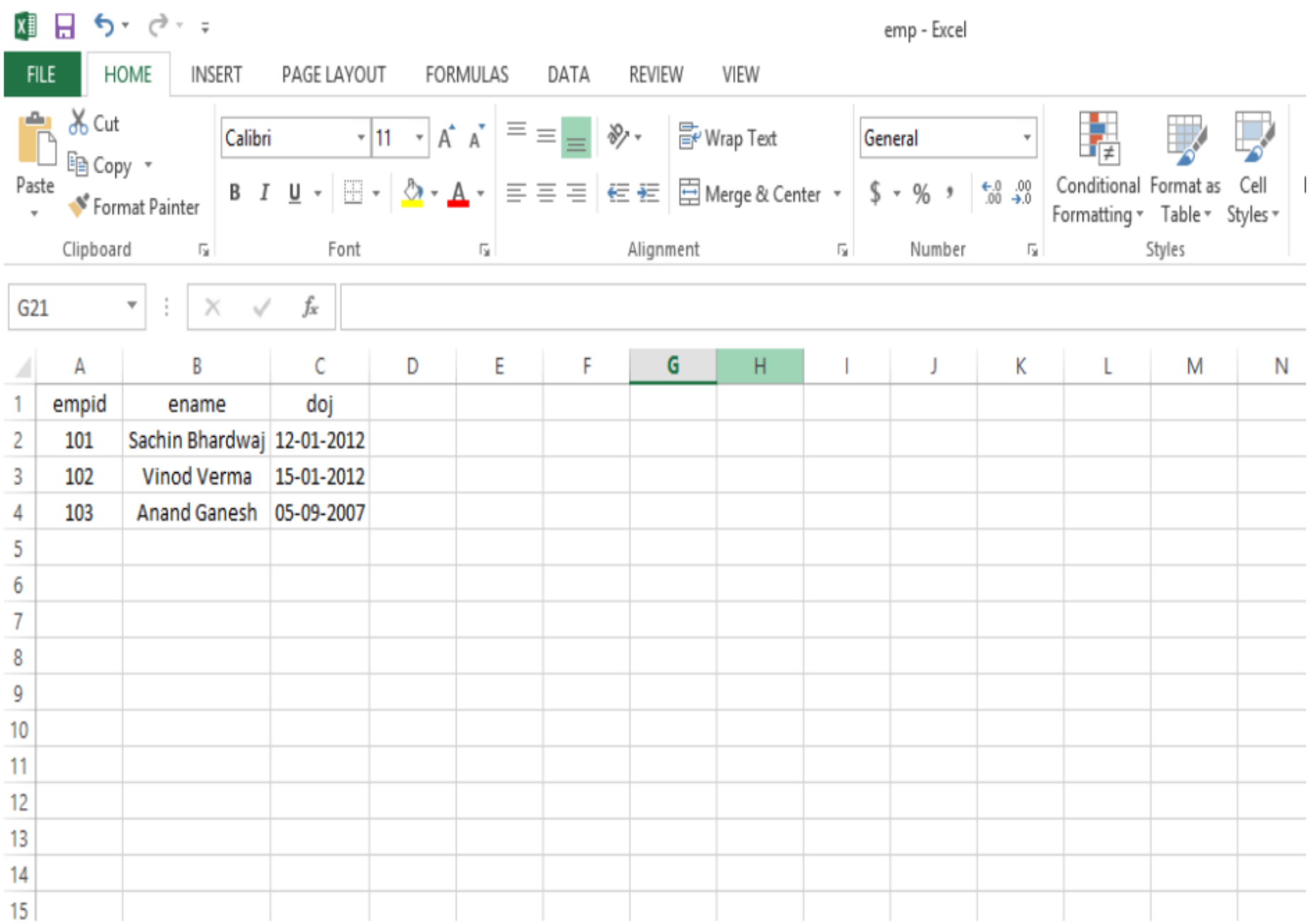
CSV is a simple file such as a spreadsheet or database.

Files in the csv format can be imported and exported from programs that store data in tables, such as Microsoft excel or Open Office.

CSV files data fields are most often separated, or delimited by a comma. Here the data in each row are delimited by comma and individual rows are separated by newline.

Pandas – CSV File

To create a csv file, first choose your favorite text editor such as-Notepad and open a new file.
Then enter the text data you want the file to contain, separating each value with a comma and each row with a new line.
Save the file with the extension.csv.
You can open the file using MS Excel or another spread sheet program. It will create the table of similar data.



	A	B	C
1	empid	ename	doj
2	101	Sachin Bhardwaj	12-01-2012
3	102	Vinod Verma	15-01-2012
4	103	Anand Ganesh	05-09-2007

pd.read_csv() method is used to read a csv file.

Pandas – CSV File

```python
# importing pandas module
import pandas as pd
# making data frame
df = pd.read_csv("E:\emp.csv")
print(df)

```

```
   empid           ename         doj
0    101  Sachin Bhardwaj  12-01-2012
1    102     Vinod Verma   15-01-2012
2    103    Anand Ganesh   05-09-2007
```

Thank You